# 21.09.2026

# RTO und RPO

# Zwei Kennzahlen für den Notfall

RTO und RPO sind zwei zentrale Kennzahlen im Notfall- und Backup-Management, die häufig verwechselt werden, aber unterschiedliche Fragen beantworten.

- RTO (Recovery Time Objective) beantwortet die Frage: Wie lange darf die Wiederherstellung dauern, bis das System wieder verfügbar ist?
- RPO (Recovery Point Objective) beantwortet die Frage: Wie viel Datenverlust ist akzeptabel, gemessen in Zeit seit der letzten Sicherung?

# Zwei Kennzahlen für den Notfall

**Beide Werte werden im Vorfeld festgelegt, meist im Rahmen eines Notfallplans oder einer Risikoanalyse.**

**Sie bestimmen direkt, welche Backup- und Wiederherstellungsstrategie ein Unternehmen benötigt.**

# RTO (Recovery Time Objective)

Die RTO gibt an, wie viel Zeit nach einem Ausfall maximal vergehen darf, bis ein System oder ein Dienst wieder einsatzbereit ist.

„Die Zeit wird ab dem Moment des Ausfalls gezählt."

Eine RTO von vier Stunden bedeutet: Sobald ein System ausfällt, es muss innerhalb von vier Stunden wieder verfügbar sein.

# RTO (Recovery Time Objective)

Die RTO bestimmt maßgeblich, welche Wiederherstellungsmechanismen nötig sind.

- Eine sehr kurze RTO (etwa wenige Minuten) erfordert automatisierte Failover-Mechanismen und redundante Systeme, die nahtlos übernehmen.
- Eine längere RTO (etwa 24 Stunden) kann auch durch manuelles Einspielen eines Backups auf neuer Hardware erreicht werden.

Je kürzer die geforderte RTO, desto teurer und komplexer die nötige Infrastruktur.

# RPO (Recovery Point Objective)

Die RPO gibt an, welcher maximale Zeitraum zwischen der letzten Sicherung und einem Ausfall liegen darf.

„Wie viele Daten dürfen verloren gehen?“

Eine RPO von 24 Stunden bedeutet:
Im Falle eines Ausfalls dürfen maximal die Daten der letzten 24 Stunden verloren sein, weil die letzte Sicherung höchstens 24 Stunden alt sein muss.

# RPO (Recovery Point Objective)

Die RPO bestimmt direkt, wie häufig Sicherungen durchgeführt werden müssen.

- Eine RPO von 24 Stunden lässt sich mit täglichen Backups erreichen.
- Eine RPO von wenigen Minuten erfordert kontinuierliche Datenreplikation, bei der Änderungen nahezu in Echtzeit auf ein zweites System übertragen werden (klassische periodische Backups reichen dafür nicht mehr aus).

# Prüfungsrelevante Einordnung



In der AP2 werden RTO und RPO meist in ein Szenario eingebettet, in dem ein Unternehmen seine Anforderungen beschreibt, und der Prüfling daraus eine passende Backup- oder Notfallstrategie ableiten soll.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen fordert, dass im Falle eines Serverausfalls maximal Daten eines Arbeitstages verloren gehen dürfen und das System innerhalb von zwei Stunden wieder verfügbar sein muss. Definieren Sie RTO und RPO für dieses Szenario und schlagen Sie eine geeignete Backup-Strategie vor.“

# Prüfungsrelevante Einordnung



Die erwartete Antwort definiert beide Begriffe korrekt, leitet aus der Aufgabenstellung konkrete Werte ab (RPO = 24 Stunden, RTO = 2 Stunden) und schlägt eine passende Lösung vor: etwa tägliche Backups kombiniert mit einem Standby-System für schnelle Wiederherstellung.

Wer RTO und RPO nur definiert, ohne sie auf das Szenario anzuwenden, lässt Punkte liegen.

# Themenblock 4
# Hochverfügbarkeit und Clustering

# Überblick

- Active/Active vs. Active/Passive Clustering
- Loadbalancing (Round Robin, Least Connections)
- Heartbeat, Split-Brain-Problem
- Failover-Szenarien und Verfügbarkeiten

Typische Prüfungslogik:

„Ein Unternehmen betreibt zwei redundante Server mit je 99% Verfügbarkeit in einem Active/Passive-Cluster. Berechnen Sie die Gesamtverfügbarkeit bei Parallelschaltung und erläutern Sie, welches Risiko beim Failover-Vorgang (Heartbeat-Ausfall) auftreten kann.“

# Active/Active vs. Active/Passive Clustering

# Was ist ein Cluster?

Ein Cluster ist ein Verbund mehrerer Server (sogenannter Knoten oder Nodes), die zusammenarbeiten, um Ausfallsicherheit und/oder Leistung zu erhöhen.

Fällt ein Knoten aus, übernehmen die verbleibenden Knoten die Aufgabe, sodass der Dienst weiterhin verfügbar bleibt.

Wie genau diese Übernahme abläuft und welche Knoten im Normalbetrieb aktiv arbeiten, unterscheidet sich grundlegend zwischen zwei Cluster-Architekturen: Active/Active und Active/Passive.

Diese Unterscheidung ist für die AP2 zentral, weil sie die Grundlage für viele weitere Konzepte wie Loadbalancing und Failover bildet.

# Active/Passive Clustering

- Bei einem Active/Passive-Cluster ist im Normalbetrieb nur ein Knoten aktiv und verarbeitet die Anfragen.
- Die anderen Knoten laufen im Standby-Modus mit, ohne aktiv Last zu übernehmen.
- Der passive Knoten überwacht den aktiven Knoten kontinuierlich.
- Fällt der aktive Knoten aus, übernimmt der passive Knoten automatisch dessen Aufgaben (Failover).

# Active/Passive Clustering

- Der Vorteil dieser Architektur ist die Einfachheit: Es gibt zu jedem Zeitpunkt nur einen Knoten, der Daten verändert, wodurch Konsistenzprobleme weitgehend vermieden werden.
- Der Nachteil ist die ineffiziente Ressourcennutzung. Der passive Knoten steht im Normalbetrieb ungenutzt bereit und verursacht trotzdem Kosten.

# Active/Active Clustering

- Bei einem Active/Active-Cluster sind alle Knoten gleichzeitig aktiv und verarbeiten parallel Anfragen.
- Die Last wird auf mehrere Knoten verteilt, typischerweise mit einem vorgeschalteten Loadbalancer.
- Fällt ein Knoten aus, übernehmen die verbleibenden Knoten automatisch dessen Anteil der Last, ohne dass ein separater Failover-Vorgang nötig ist.
- Der Dienst läuft einfach mit reduzierter Kapazität weiter.

# Active/Active Clustering

- Der große Vorteil ist die effiziente Ressourcennutzung: Alle Knoten tragen im Normalbetrieb zur Leistung bei, und es gibt keine ungenutzte Standby-Hardware.
- Der Nachteil ist die höhere Komplexität. Da mehrere Knoten gleichzeitig auf gemeinsame Daten zugreifen und diese verändern können, muss die Konsistenz der Daten zwischen den Knoten sichergestellt werden, etwa durch Datenbankreplikation oder verteilte Sperrmechanismen.

# Direkter Vergleich

**Arbeiten im Normalbetrieb alle Knoten gleichzeitig, oder steht ein Teil der Knoten nur bereit?**

- Bei Active/Passive arbeitet nur ein Knoten, der Rest wartet.
- Bei Active/Active arbeiten alle Knoten gleichzeitig.

# Direkter Vergleich

Daraus ergeben sich die jeweiligen Eigenschaften:

- Active/Passive ist einfacher zu implementieren, hat aber ungenutzte Kapazität und einen kurzen Unterbrechungsmoment während des Failovers.
- Active/Active nutzt die Hardware vollständig aus und bietet bei Ausfall eines Knotens einen nahtlosen Übergang ohne spürbare Unterbrechung. Aber es erfordert eine Architektur, die mit gleichzeitigen Zugriffen mehrerer Knoten umgehen kann.

# Typische Einsatzszenarien

- Ein Datenbankserver, bei dem Datenkonsistenz oberste Priorität hat und ein kurzer Failover-Moment im Notfall akzeptabel ist, wird häufig als Active/Passive-Cluster betrieben, etwa mit einem primären und einem sekundären Datenbankserver, der die Daten per Replikation synchron hält.
- Ein Webserver-Cluster, der eine vielbesuchte Website ausliefert, profitiert dagegen von Active/Active: Alle Server liefern gleichzeitig Inhalte aus, ein Loadbalancer verteilt die Anfragen, und bei Ausfall eines Servers merken die Nutzer im Idealfall nichts, weil die übrigen Server die Last einfach mitübernehmen.

Die Wahl hängt also stark davon ab, ob der Dienst zustandslos ist (gut für Active/Active) oder einen konsistenten, gemeinsamen Datenbestand erfordert (oft eher Active/Passive).

# Prüfungsrelevante Einordnung



In der AP2 wird häufig ein Szenario beschrieben, bei dem ein Unternehmen einen Dienst hochverfügbar betreiben möchte, und der Prüfling soll begründen, welche Cluster-Architektur geeignet ist.

Eine typische Frage lautet:

„Ein Unternehmen betreibt einen Webshop und möchte sicherstellen, dass dieser auch bei Ausfall eines Servers ohne Unterbrechung erreichbar bleibt. Welche Cluster-Architektur empfehlen Sie, und warum?“

# Prüfungsrelevante Einordnung



Die erwartete Antwort nennt Active/Active, begründet dies mit der gleichzeitigen Nutzung aller Knoten und dem nahtlosen Übergang bei Ausfall, und grenzt es von Active/Passive ab, bei dem ein kurzer Failover-Moment entstehen würde.

Wichtig ist, dass die Begründung sich auf die konkreten Anforderungen des Szenarios bezieht und nicht nur die Begriffe definiert.

# Loadbalancing

# Was ist Loadbalancing?

Loadbalancing bezeichnet die Verteilung eingehender Anfragen auf mehrere Server, damit kein einzelner Server überlastet wird, während andere ungenutzt bleiben.

Ein Loadbalancer sitzt dabei als vermittelnde Instanz zwischen den Clients und den Servern: Anfragen kommen beim Loadbalancer an, dieser entscheidet nach einem bestimmten Algorithmus, an welchen Server die Anfrage weitergeleitet wird, und leitet sie entsprechend weiter.

# Was ist Loadbalancing?

Für den Client ist dieser Vorgang transparent, denn er adressiert nur den Loadbalancer und merkt nicht, welcher konkrete Server seine Anfrage tatsächlich bearbeitet.

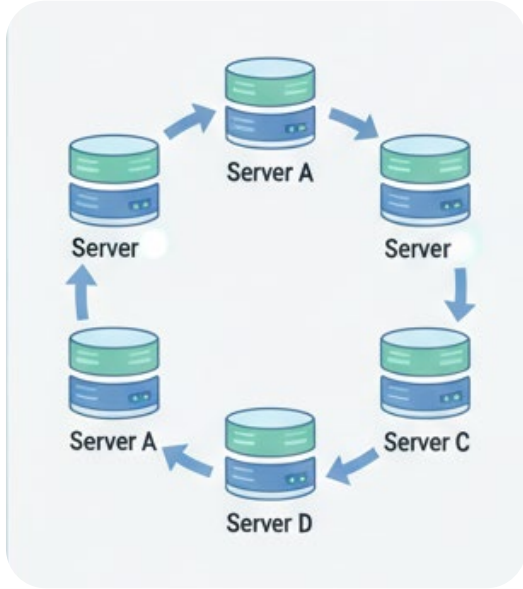
Loadbalancing ist die technische Grundlage, die Active/Active-Cluster überhaupt erst praktikabel macht.

# Round Robin

Round Robin ist der einfachste Loadbalancing-Algorithmus.

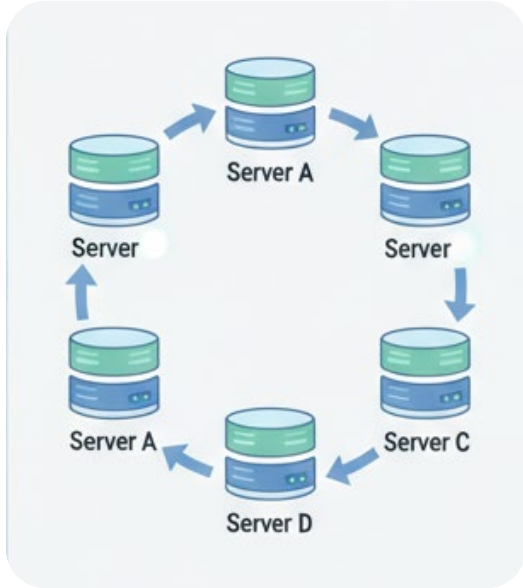
Anfragen werden der Reihe nach, zyklisch, an die verfügbaren Server verteilt.

Die erste Anfrage geht an Server 1, die zweite an Server 2, die dritte an Server 3, die vierte wieder an Server 1, und so weiter.



# Round Robin

- Der Vorteil ist auch hier die Einfachheit: die Implementierung ist unkompliziert und die Verteilung ist im Durchschnitt gleichmäßig.
- Der Nachteil: Round Robin berücksichtigt nicht, wie ausgelastet ein Server aktuell tatsächlich ist oder wie lange eine einzelne Anfrage zur Bearbeitung braucht. Ein Server, der gerade eine besonders rechenintensive Anfrage bearbeitet, bekommt trotzdem die nächste Anfrage in der Reihe zugewiesen, wodurch sich Last ungleich verteilen kann, obwohl der Algorithmus formal „gleich“ verteilt.

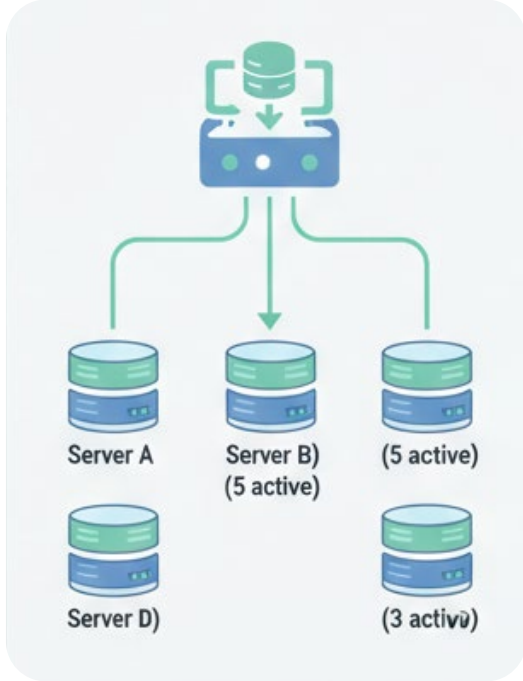


# Least Connections

Least Connections verfolgt einen dynamischeren Ansatz.

Der Loadbalancer leitet jede neue Anfrage an den Server weiter, der aktuell die wenigsten offenen Verbindungen hat.

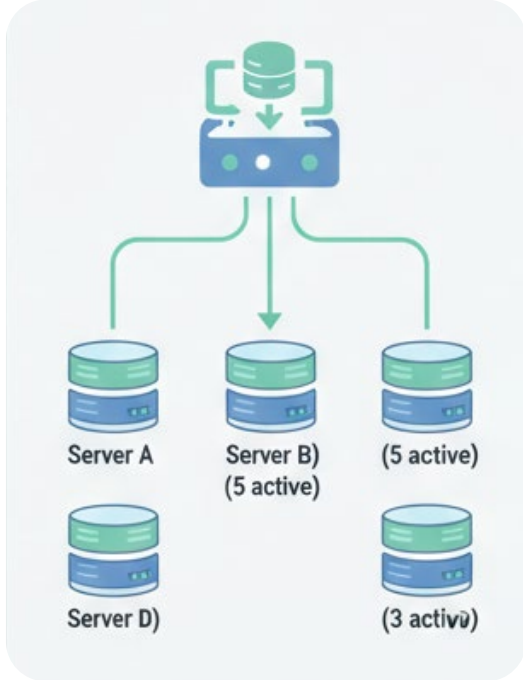
Damit wird berücksichtigt, dass nicht jede Anfrage gleich viel Last erzeugt: ein Server mit vielen lang laufenden Verbindungen bekommt seltener neue Anfragen zugewiesen als ein Server, dessen Verbindungen schnell abgeschlossen werden.



# Least Connections

Dieser Algorithmus passt sich also an die tatsächliche aktuelle Auslastung an, statt blind nach einem festen Schema zu verteilen.

- Der Vorteil ist eine realistischere Lastverteilung, besonders wenn Anfragen sehr unterschiedlich lange dauern.
- Der Nachteil ist ein etwas höherer Aufwand für den Loadbalancer, da er den Zustand jedes Servers kontinuierlich überwachen muss.



# Loadbalancing vs. Hochverfügbarkeit

Ein Irrtum wäre, Loadbalancing und Hochverfügbarkeit (HA) gleichzusetzen. Beide hängen zusammen, sind aber konzeptionell unterschiedlich.

- Loadbalancing hat primär das Ziel, Last gleichmäßig zu verteilen und damit die Performance und Skalierbarkeit zu verbessern.
- Hochverfügbarkeit hat das Ziel, den Ausfall einzelner Komponenten abzufangen, damit der Dienst insgesamt verfügbar bleibt.

# Prüfungsrelevante Einordnung



In der AP2 wird häufig nach dem Unterschied zwischen Loadbalancing und Hochverfügbarkeit gefragt, oder es wird ein Szenario beschrieben, in dem ein geeigneter Loadbalancing-Algorithmus empfohlen werden soll.

Eine typische Aufgabe könnte lauten:

„Ein Unternehmen betreibt drei Webserver mit identischer Hardware.
Die Anfragen unterscheiden sich stark in ihrer Bearbeitungsdauer.
Welcher Loadbalancing-Algorithmus ist geeignet, und warum?“

# Prüfungsrelevante Einordnung



Die erwartete Antwort empfiehlt Least Connections, weil dieser Algorithmus die tatsächliche Auslastung berücksichtigt.

Im Gegensatz zu Round Robin, der bei unterschiedlich langen Anfragen zu ungleicher Lastverteilung führen würde.

Wichtig ist außerdem, den Loadbalancer selbst als potenziellen Single Point of Failure zu erkennen und idealerweise zu erwähnen, dass auch er redundant ausgelegt werden sollte.

# Heartbeat, Split-Brain-Problem

# Wie weiß ein Cluster, ob ein Knoten noch lebt?

Damit ein Cluster auf den Ausfall eines Knotens reagieren kann (etwa durch Failover bei Active/Passive oder durch Umverteilung der Last bei Active/Active) muss er zunächst feststellen, dass ein Knoten überhaupt ausgefallen ist.

Diese Überwachung übernimmt der sogenannte Heartbeat-Mechanismus.

# Wie weiß ein Cluster, ob ein Knoten noch lebt?

Ohne verlässliche Erkennung eines Ausfalls kann kein Failover stattfinden.

Eine fehlerhafte Erkennung kann zu einem der gefährlichsten Probleme in Cluster-Umgebungen führen, dem sogenannten Split-Brain.

# Das Heartbeat-Konzept

Ein Heartbeat ist ein regelmäßiges, kleines Signal, das die Knoten eines Clusters sich gegenseitig senden, um ihre Verfügbarkeit zu bestätigen, vergleichbar mit einem „Ich bin noch da“-Signal in kurzen Zeitabständen, etwa alle paar Sekunden.

Bleibt dieses Signal von einem Knoten über einen definierten Zeitraum aus, gehen die übrigen Knoten davon aus, dass dieser Knoten ausgefallen ist, und leiten entsprechende Maßnahmen ein (bspw. einen Failover).

# Das Heartbeat-Konzept

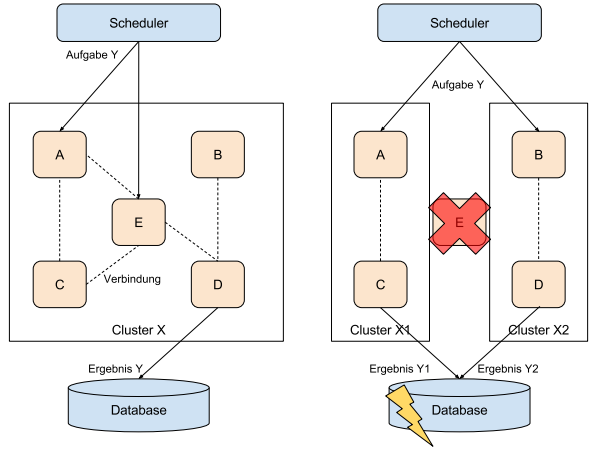
Heartbeat-Signale werden typischerweise über eine eigene, dedizierte Netzwerkverbindung zwischen den Knoten übertragen, getrennt vom normalen Datenverkehr.

Diese Trennung ist wichtig, damit eine Überlastung des normalen Netzwerks nicht fälschlicherweise als Knotenausfall interpretiert wird.

# Das Split-Brain-Problem

Ein Split-Brain-Szenario entsteht, wenn die Heartbeat-Verbindung zwischen den Knoten unterbrochen wird, obwohl die Knoten selbst aber weiterhin einwandfrei laufen und arbeiten.

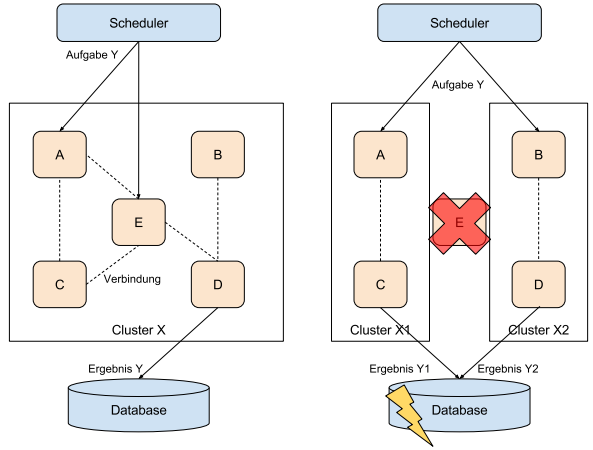
Jeder Knoten registriert: „Ich bekomme kein Heartbeat-Signal mehr vom anderen Knoten. Er muss ausgefallen sein.“



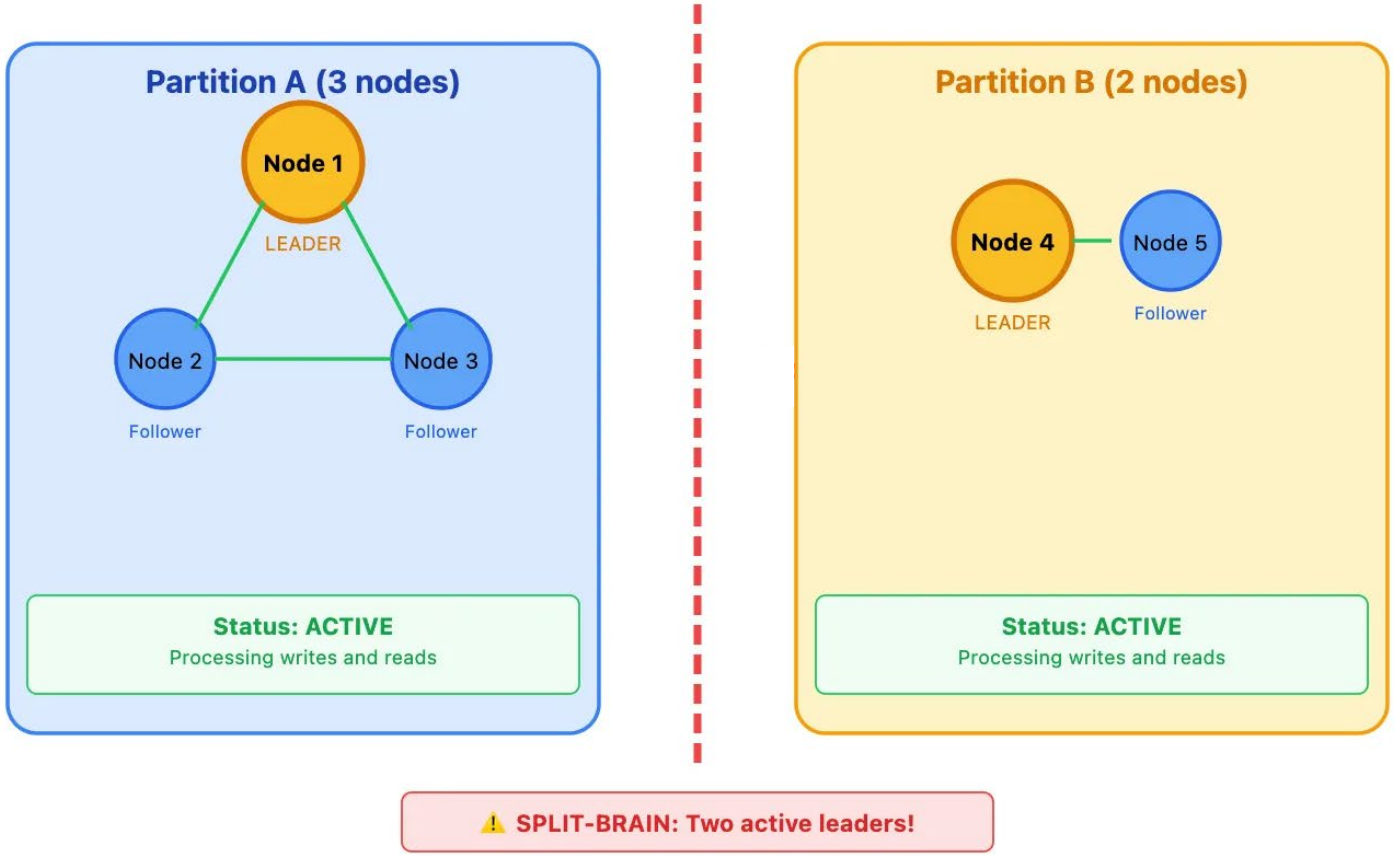
# Das Split-Brain-Problem

In Wirklichkeit ist aber nur die Verbindung zwischen den Knoten unterbrochen, nicht die Knoten selbst. Beide Knoten halten sich nun jeweils für den einzig verbliebenen aktiven Knoten und übernehmen (insbesondere in Active/Passive-Clustern) beide die aktive Rolle gleichzeitig.

Das Cluster ist nun „gespalten“: zwei Knoten, die unabhängig voneinander glauben, allein zuständig zu sein.



# Das Split-Brain-Problem

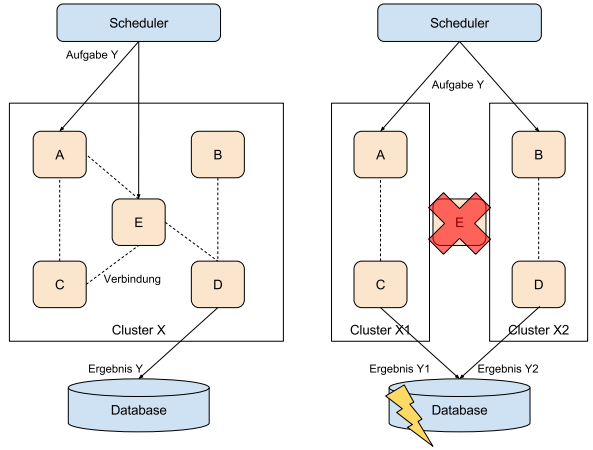


# Warum Split-Brain so gefährlich ist

Die eigentliche Gefahr entsteht, wenn beide Knoten in dieser Situation auf gemeinsame Daten (etwa eine Datenbank oder ein gemeinsames Speichersystem ) zugreifen und diese verändern.

Beide Knoten nehmen Schreibvorgänge an, ohne von den Änderungen des anderen Knotens zu wissen.

Das Ergebnis sind widersprüchliche, inkonsistente Datenstände (s. g. Datenkorruption).

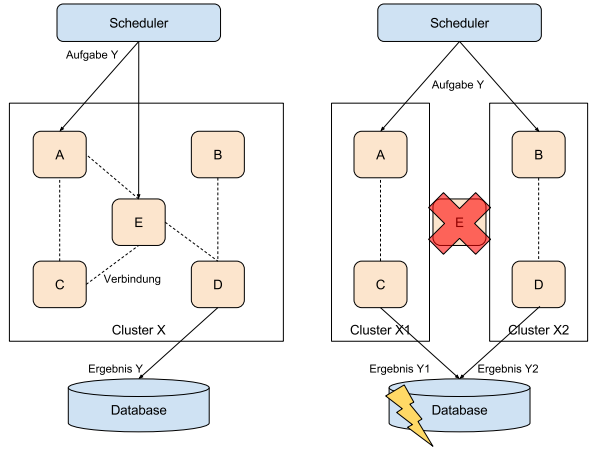


# Warum Split-Brain so gefährlich ist

Im schlimmsten Fall überschreiben sich die Knoten gegenseitig Daten, ohne dass dies bemerkt wird, bis die Verbindung wiederhergestellt ist und der Widerspruch sichtbar wird.

Zu diesem Zeitpunkt ist oft nicht mehr nachvollziehbar, welcher Datenstand korrekt ist.

Die Wiederherstellung eines konsistenten Zustands kann erhebliche manuelle Aufwände erfordern oder sogar Datenverlust bedeuten.



# Maßnahmen gegen Split-Brain

Um Split-Brain zu vermeiden, setzen Cluster-Systeme verschiedene Schutz-mechanismen ein.

- Redundante Heartbeat-Verbindungen
- Quorum-Mechanismus
- Fencing (bzw. „STONITH“)

Der Einsatz redundanter Heartbeat-Verbindungen (etwa über zwei getrennte Netzwerkpfade) sorgt dafür, dass der Ausfall einer einzelnen Verbindung nicht sofort als Knotenausfall interpretiert wird.

# Maßnahmen gegen Split-Brain

Um Split-Brain zu vermeiden, setzen Cluster-Systeme verschiedene Schutz-mechanismen ein.

- Redundante Heartbeat-Verbindungen
- Quorum-Mechanismus
- Fencing (bzw. „STONITH“)

Ein dritter, unabhängiger Knoten oder ein spezieller „Witness“-Server entscheidet im Zweifelsfall, welcher der beiden Knoten aktiv bleiben darf (typischerweise der, der noch Kontakt zu diesem Quorum-Knoten hat).

# Maßnahmen gegen Split-Brain

Um Split-Brain zu vermeiden, setzen Cluster-Systeme verschiedene Schutz-mechanismen ein.

- Redundante Heartbeat-Verbindungen
- Quorum-Mechanismus
- Fencing (bzw. „STONITH“)

## Fencing

(auch STONITH genannt: "Shoot The Other Node In The Head").

Im Zweifelsfall wird der vermeintlich aus-gefallene Knoten aktiv vom gemeinsamen Speicher oder Netzwerk getrennt oder sogar zwangsweise heruntergefahren, um sicherzustellen, dass er keine Schreib-zugriffe mehr durchführen kann.

# Prüfungsrelevante Einordnung



In der AP2 wird das Split-Brain-Problem meist konzeptionell abgefragt. Es wird nicht erwartet, dass Fencing-Mechanismen konfiguriert werden, sondern dass das Problem verstanden und erklärt werden kann.

Eine typische Aufgabe könnte lauten:
„Erklären Sie, was unter einem Split-Brain-Szenario in einem Cluster zu verstehen ist, wie es entstehen kann und welche Folgen es haben kann.“

# Prüfungsrelevante Einordnung



Die erwartete Antwort beschreibt den Ausfall der Heartbeat-Verbindung als Ursache, erklärt, dass beide Knoten sich fälschlicherweise für aktiv halten, und nennt als Folge die Gefahr inkonsistenter Daten bei gemeinsamem Datenzugriff.

Wer zusätzlich eine Schutzmaßnahme wie Quorum oder Fencing benennen kann, zeigt vertieftes Verständnis und hebt sich von einer reinen Definitionsantwort ab.

# Failover-Szenarien

# Was ist Failover?

Failover bezeichnet den automatischen oder manuellen Übergang der Funktion eines ausgefallenen Systems auf ein redundantes Ersatzsystem (Hochverfügbarkeit).

Das Ziel ist, die Verfügbarkeit eines Dienstes trotz eines Ausfalls aufrechtzuerhalten.

- Cluster, Heartbeat-Überwachung und redundante Systeme sind die Bausteine.
- Failover ist der Vorgang, der im Ernstfall tatsächlich greift.

Für die AP2 ist wichtig, verschiedene Failover-Szenarien nicht nur zu kennen, sondern auch deren Ablauf und die Konsequenzen (insbesondere Unterbrechungszeit und Datenverlust) beschreiben zu können.

# Failover bei Active/Passive

1. Der passive Knoten überwacht den aktiven Knoten per Heartbeat.
2. Bleibt das Heartbeat-Signal über einen definierten Schwellenwert hinweg aus, geht der passive Knoten davon aus, dass der aktive Knoten ausgefallen ist.
3. Er übernimmt nun die Rolle des aktiven Knotens (das umfasst typischerweise die Übernahme einer virtuellen IP-Adresse, unter der der Dienst erreichbar ist, sowie den Zugriff auf gemeinsam genutzten Speicher oder die zuletzt replizierten Daten).
4. Während dieses Übergangs entsteht eine kurze Unterbrechung (die s. g. Failover-Zeit), in der der Dienst nicht erreichbar ist.

# Failover bei Active/Active

1. Der Loadbalancer erkennt, dass ein Knoten nicht mehr antwortet, und leitet keine weiteren Anfragen mehr an ihn weiter.
2. Die übrigen Knoten verarbeiten die Last, die zuvor auf alle Knoten verteilt war.
3. Für Nutzer, deren Anfrage gerade an den ausgefallenen Knoten ging, kann es zu einer einzelnen fehlgeschlagenen Anfrage kommen.
4. Neue Anfragen werden aber sofort an funktionierende Knoten geleitet.
5. Dieser Übergang ist in der Regel deutlich schneller und weniger spürbar als bei Active/Passive.

# Failback: Die Rückkehr zum Normalzustand

## Active/Passive

- Der ursprünglich aktive Knoten wird wiederhergestellt, übernimmt aber nicht automatisch sofort wieder die aktive Rolle zurück, denn das würde eine erneute Unterbrechung bedeuten.
- Failback wird häufig manuell und zu einem geplanten Zeitpunkt durchgeführt, etwa außerhalb der Geschäftszeiten, um eine zweite Unterbrechung kontrolliert zu steuern.
- Vor dem Failback müssen alle Daten, die während des Failovers auf dem Ersatzsystem entstanden sind, mit dem wiederhergestellten System synchronisiert werden, um eventuelle Datenverluste oder Inkonsistenzen zu vermeiden.

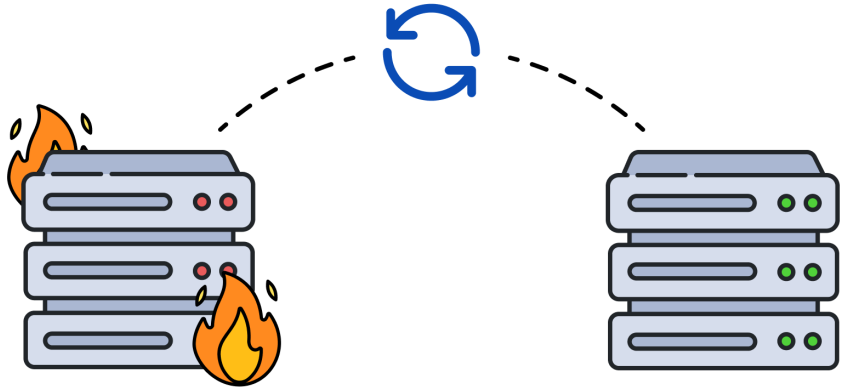
# Failback: Die Rückkehr zum Normalzustand

## Active/Active

- Dieser Fall ist einfacher.
- Der zurückkehrende Knoten muss sich vor der Wiederaufnahme aber mit dem aktuellen Datenstand synchronisieren (Resynchronisation), da er während seines Ausfalls Änderungen „verpasst“ hat.
- Erst wenn er konsistent ist, nimmt er wieder Traffic an.
- Das geschieht idealerweise unterbrechungsfrei, indem der Knoten schrittweise wieder in die Lastverteilung genommen wird.

# Failover über Standorte hinweg: Disaster Recovery

- Failover findet nicht nur innerhalb eines Rechenzentrums zwischen zwei Knoten statt.
- Es kann auch standortübergreifend erfolgen, etwa wenn ein gesamtes Rechenzentrum ausfällt (Stromausfall, Naturkatastrophe, Brand).
- In diesem Fall übernimmt ein zweites Rechenzentrum an einem anderen Standort die Funktion.
- Dieses Szenario wird als Disaster Recovery bezeichnet.



# Failover über Standorte hinweg: Disaster Recovery

Die Anforderungen sind hier deutlich höher:

- Die Daten müssen kontinuierlich zwischen den Standorten repliziert werden.
- Die Failover-Zeit ist meist länger als bei lokalem Clustering, weil mehr Komponenten (Netzwerk, DNS, Anwendungen) auf den neuen Standort umgestellt werden müssen.

Hier kommen die zuvor behandelten Konzepte RTO und RPO wieder direkt zum Einsatz, denn sie definieren, wie schnell und mit wie viel Datenverlust dieser standortübergreifende Failover ablaufen darf.

# Prüfungsrelevante Einordnung



In der AP2 wird häufig ein konkretes Ausfallszenario beschrieben, und der Prüfling soll den Failover-Ablauf erklären, inklusive der Konsequenzen für Verfügbarkeit und Daten.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen betreibt zwei Datenbankserver im Active/Passive-Modus. Der aktive Server fällt aus. Beschreiben Sie den Ablauf des Failovers und gehen Sie auf mögliche Auswirkungen für die Nutzer ein.“

# Prüfungsrelevante Einordnung



Die erwartete Antwort beschreibt die Heartbeat-Erkennung, die Übernahme durch den passiven Knoten inklusive virtueller IP, die kurze Unterbrechung während des Übergangs, und idealerweise auch den späteren Failback-Vorgang.

Wer den gesamten Lebenszyklus (Ausfall, Erkennung, Übernahme, spätere Rückkehr) beschreiben kann, zeigt vollständiges Verständnis des Konzepts.

# Verfügbarkeit berechnen

# Einleitung

- Bisher haben wir die Verfügbarkeit eines einzelnen Systems betrachtet, also bspw. Einen einzelnen Server mit einem SLA von 99,9%.
- In der Praxis bestehen IT-Infrastrukturen aber selten aus einem einzelnen System, sondern aus mehreren Komponenten, die zusammenwirken.

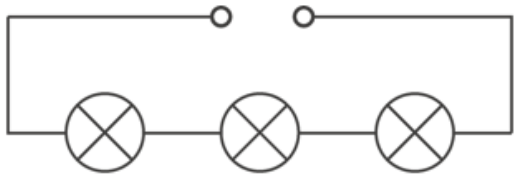
# Einleitung

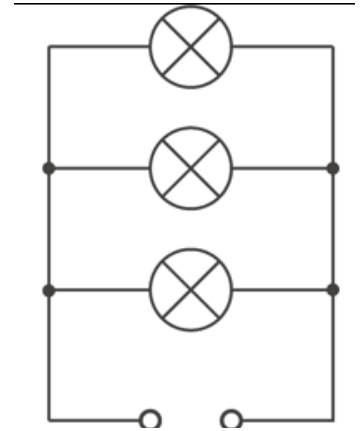
**Wie verändert sich die Gesamtverfügbarkeit, wenn mehrere Systeme zusammenarbeiten?**

# Reihenschaltung vs. Parallelschaltung

Die Antwort hängt davon ab, wie sie zusammenarbeiten.

- Voneinander abhängig: Reihenschaltung
- Sich gegenseitig absichern: Parallelschaltung





Dieser Unterschied ist mathematisch einfach, aber konzeptionell entscheidend und ein klassischer Rechenaufgabentyp in der AP2.

# Reihenschaltung: Verfügbarkeit sinkt

**Wenn zwei Systeme in Reihe geschaltet sind (das heißt, beide müssen funktionieren, damit der Gesamtdienst funktioniert), multiplizieren sich ihre Verfügbarkeiten.**

# Reihenschaltung: Verfügbarkeit sinkt

„Eine Kette ist nur so stark wie ihr schwächstes Glied“

- Ein Webserver greift auf einen Datenbankserver zu.
- Funktioniert der Datenbankserver nicht, ist der Webdienst insgesamt nicht verfügbar, selbst wenn der Webserver einwandfrei läuft.
- Hat der Webserver eine Verfügbarkeit von 99% und der Datenbankserver ebenfalls 99%, beträgt die Gesamtverfügbarkeit 99% × 99% = 98,01%.
- Das Ergebnis ist niedriger als die Verfügbarkeit jeder einzelnen Komponente.

Reihenschaltung:
Je mehr Komponenten in der Kette, desto geringer die Gesamtverfügbarkeit.

# Parallelschaltung: Verfügbarkeit steigt

**Bei einer Parallelschaltung sichern sich zwei Systeme gegenseitig ab: der Gesamtdienst ist bereits dann verfügbar, wenn mindestens eines der beiden Systeme funktioniert.**

# Parallelschaltung: Verfügbarkeit steigt

„Redundanz und Clustering“

- Zwei redundante Server (also bspw. ein Cluster mit zwei Knoten oder zwei unabhängige Internetanbindungen) sind klassische Beispiele für Parallelschaltung.
- Bei dieser Konstellation steigt die Gesamtverfügbarkeit über die Verfügbarkeit der einzelnen Komponenten hinaus, denn ein Ausfall führt nur dann zum Gesamtausfall, wenn beide Systeme gleichzeitig ausfallen.
- Und die Wahrscheinlichkeit, dass beide Systeme gleichzeitig ausfallen, ist deutlich geringer als die Wahrscheinlichkeit, dass eines von beiden ausfällt.

# Die Formel für Parallelschaltung

- Um die Gesamtverfügbarkeit bei Parallelschaltung zu berechnen, rechnet man am einfachsten über die Nichtverfügbarkeit (also die Ausfallwahrscheinlichkeit).
- Die Nichtverfügbarkeit eines Systems ergibt sich aus 100% minus seiner Verfügbarkeit.
- Bei zwei parallelen Systemen tritt ein Gesamtausfall nur ein, wenn beide Systeme gleichzeitig ausfallen: die Ausfall-Wahrscheinlichkeiten dafür werden multipliziert.
- Die Formel lautet also: Gesamtnichtverfügbarkeit = Nichtverfügbarkeit von System 1 × Nichtverfügbarkeit von System 2.

Gesamtverfügbarkeit = 100% minus Gesamt-Nichtverfügbarkeit.

# Die Formel für Parallelschaltung

**Der Ansatz „über die Ausfallwahrscheinlichkeit statt über die Verfügbarkeit“ selbst zu rechnen ist der Schlüssel zu diesem Aufgabentyp.**

# Beispiel: zwei Systeme mit je 99%

- Zwei redundante Systeme haben jeweils eine Verfügbarkeit von 99%.
- Die Nichtverfügbarkeit jedes einzelnen Systems beträgt damit 1% bzw. 0,01
- Bei Parallelschaltung tritt ein Gesamtausfall nur ein, wenn beide Systeme gleichzeitig ausfallen: 0,01 × 0,01 = 0,0001, also 0,01%.
- Die Gesamtverfügbarkeit beträgt damit 100% − 0,01% = 99,99%.

# Beispiel: zwei Systeme mit je 99%

**Aus zwei Systemen mit jeweils 99% Verfügbarkeit ("zwei Neunen") entsteht durch Redundanz eine Gesamtverfügbarkeit von 99,99% ("vier Neunen").**

**Das zeigt, warum Redundanz ein so wirkungsvolles Mittel zur Erhöhung der Verfügbarkeit ist.**

# Prüfungsrelevante Einordnung



In der AP2 wird dieser Rechenaufgabentyp häufig mit den vorherigen Themen kombiniert, etwa mit Clustering oder SLA-Verfügbarkeits-klassen.

Eine typische Aufgabe könnte lauten:

„Ein Unternehmen betreibt zwei Server mit jeweils einer Verfügbarkeit von 99% in einem Active/Active-Cluster. Berechnen Sie die Gesamtverfügbarkeit des Systems und ordnen Sie das Ergebnis in eine Verfügbarkeitsklasse ein.“

# Prüfungsrelevante Einordnung



Die erwartete Antwort rechnet über die Nichtverfügbarkeiten (0,01 × 0,01 = 0,0001), kommt auf 99,99% Gesamtverfügbarkeit und ordnet das Ergebnis korrekt als "vier Neunen" ein mit einer maximal zulässigen Ausfallzeit von etwa 52 Minuten pro Jahr.

Wichtig ist außerdem die konzeptionelle Erkenntnis, die oft als Zusatzfrage kommt: Reihenschaltung verschlechtert die Gesamt-verfügbarkeit, Parallelschaltung (also Redundanz) verbessert sie.

Wer diesen Zusammenhang einmal verstanden hat, kann beliebige Kombinationen aus Verfügbarkeitswerten durchrechnen.

# Themenblock 5
# Unterbrechungsfreie Stromversorgung

# Einleitung

- Server, Netzwerkgeräte und Speichersysteme sind auf eine stabile Stromversorgung angewiesen.
- Fällt der Strom aus, drohen Datenverlust und Hardwareschäden durch unsauberes Abschalten und ungeplante Ausfallzeiten.
- Eine Unterbrechungsfreie Stromversorgung (USV) schützt vor genau diesen Szenarien, indem sie bei einem Stromausfall nahtlos auf Batteriebetrieb umschaltet.
- Damit überbrückt sie die Zeit bis zur Wiederherstellung der Stromversorgung oder bis ein Notstromaggregat anläuft, ohne dass die angeschlossenen Geräte auch nur einen Moment ohne Strom sind.

# Einleitung

- Eine USV schützt nicht nur vor vollständigen Stromausfällen, sondern auch vor Spannungsschwankungen, Überspannungen und Netzstörungen, die empfindliche IT-Hardware beschädigen können.
- Sie filtert die eingehende Netzspannung und gibt eine saubere, stabile Spannung an die angeschlossenen Geräte weiter.
- In Rechenzentren ist die USV daher nicht nur ein Notfallgerät, sondern ein dauerhaft aktiver Bestandteil der Stromversorgungs-infrastruktur.

# Offline (Standby)

- Die einfachste USV-Variante ist die Offline-USV. Im Normalbetrieb fließt der Strom direkt vom Netz zu den Geräten, die Batterie ist nicht im Stromkreis.
- Fällt die Netzspannung aus, schaltet die USV in wenigen Millisekunden auf Batteriebetrieb um.
- Nachteil: In dieser kurzen Umschaltzeit gibt es eine minimale Unterbrechung. Das ist für die meisten Geräte unproblematisch, für sehr empfindliche Systeme aber möglicherweise kritisch.
- Vorteil: Offline-USVs sind kostengünstig.
- Einsatz: Offline-USVs eignen sich für einfache Arbeitsplatzrechner oder unkritische Geräte.

# Line-Interactive

- Die Line-Interactive-USV ist die am weitesten verbreitete Variante im professionellen Umfeld. Sie enthält einen Spannungsregler (AVR — Automatic Voltage Regulation), der Spannungsschwankungen ausgleicht, ohne auf Batteriebetrieb umschalten zu müssen.
- Nur bei vollständigem Stromausfall oder extremen Spannungsabweichungen wird die Batterie aktiviert.
- Vorteil: Die Umschaltzeit ist sehr kurz und für die meisten IT-Geräte unproblematisch.
- Einsatz: Line-Interactive-USVs bieten ein gutes Preis-Leistungs-Verhältnis für Server und Netzwerkgeräte in kleineren bis mittleren Umgebungen.

# Online (Doppelwandler)

- Die Online-USV (auch Doppelwandler-USV genannt) bietet den höchsten Schutz.
- Der Strom wird permanent zunächst in Gleichstrom umgewandelt (zum Laden der Batterie) und dann wieder in sauberen Wechsel-strom für die angeschlossenen Geräte.
- Die Geräte laufen also immer über die Batterie, daher gibt es keine Umschaltzeit, da die Batterie permanent im Stromkreis ist.
- Vorteil: Fällt das Netz aus, merken die angeschlossenen Geräte davon schlicht nichts.
- Nachteil: Online-USVs sind teurer und erzeugen mehr Wärme.
- Einsatz: Sie sind aber die einzige Wahl für hochkritische Systeme wie Datenbankserver oder Rechenzentren.

# Dimensionierung

## Wie groß muss eine USV sein?

- Die Größe einer USV wird in VA (Voltampere) oder Watt angegeben.
- Sie muss ausreichend Kapazität haben, um alle angeschlossenen Geräte für eine definierte Überbrückungszeit zu versorgen.
- Für die Dimensionierung addiert man die Leistungsaufnahme aller anzuschließenden Geräte und multipliziert mit einem Sicherheitsfaktor.
- Gewünschte Überbrückungszeit: Soll die USV nur Zeit für ein geordnetes Herunterfahren der Systeme (typischerweise 5-15 Minuten) überbrücken, oder muss sie bis zum Anlaufen eines Notstromaggregats (30 Minuten oder mehr) durchhalten?
- Je länger die Überbrückungszeit, desto größer die erforderliche Batteriekapazität.

# Beispiel

Angeschlossene Geräte:

- Server: 400 W
- Switch: 100 W
- Monitor: 50 W

| | | |
|---|---|---|
| Summe Leistungsaufnahme: | 550 W | |
| Sicherheitsfaktor (z. B. 1,3 für Reserve/Alterung): | 550 W × 1,3 | = 715 W |
| Umrechnung in VA (bei angenommenem Leistungsfaktor 0,8): | 715 W ÷ 0,8 | ≈ 894 VA |

Man würde eine USV mit mind. 900–1000 VA wählen.

# Prüfungsrelevante Einordnung



In der AP2 wird die USV meist als Planungs- oder Vergleichsaufgabe geprüft.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen möchte seinen Datenbankserver vor Stromausfällen und Spannungsschwankungen schützen. Welchen USV-Typ empfehlen Sie, und warum?“

# Prüfungsrelevante Einordnung



Die erwartete Antwort empfiehlt die Online-USV (Doppelwandler), begründet mit der fehlenden Umschaltzeit und dem permanenten Batteriebetrieb, denn für einen Datenbankserver ist jede auch noch so kurze Unterbrechung kritisch.

Wer zusätzlich die drei Typen sauber abgrenzt und zusätzlich die Dimensionierung nach Leistungsaufnahme und Überbrückungszeit erwähnt, zeigt vollständiges Verständnis.

21.09.2026

# Ende